

EECS 482

Introduction to operating systems

CPU scheduling

Peter Chen
University of Michigan

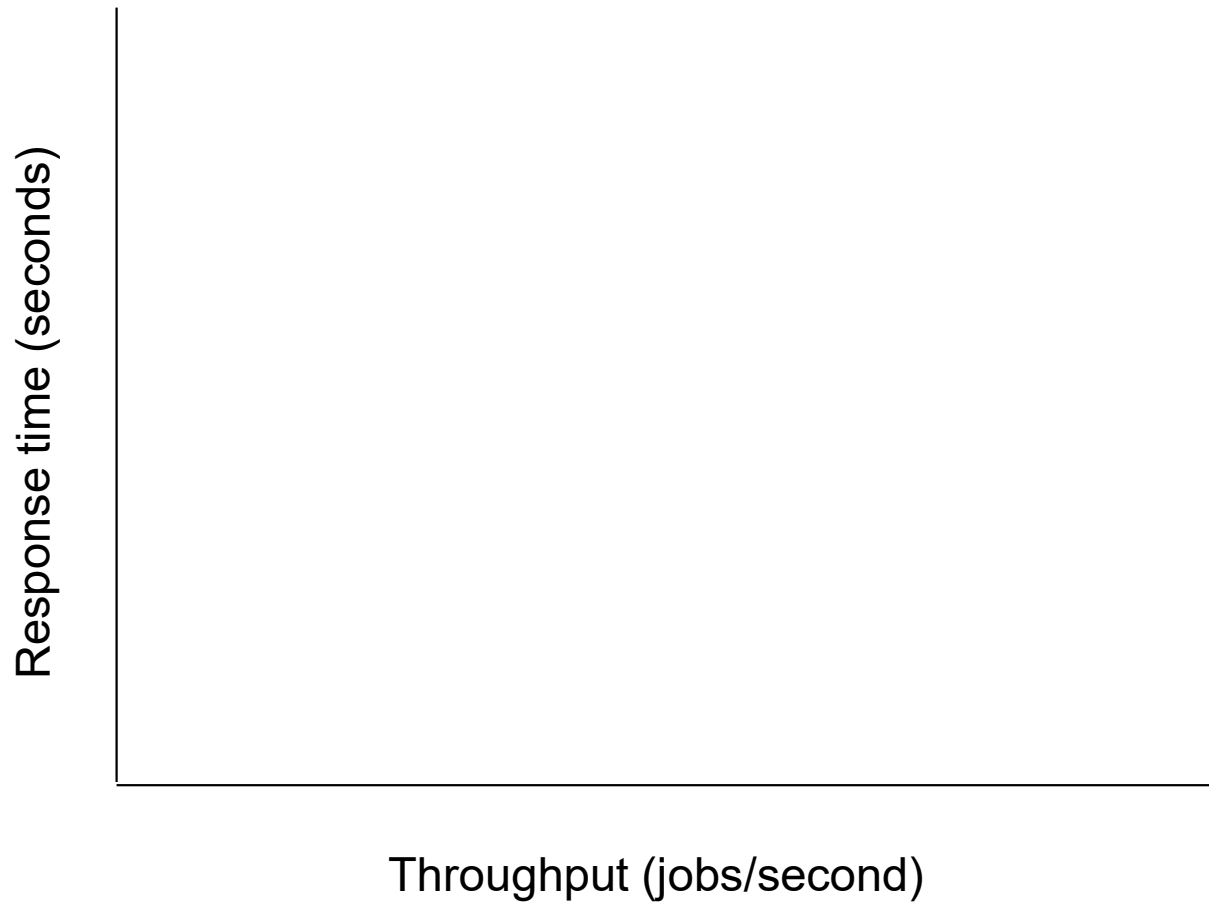
CPU scheduling

- “If >1 thread is ready, choose which thread to run”
- How to choose
 - Project 2 and 3: FIFO
 - Many other policies possible
- Goals of CPU scheduling
 - Performance
 - Fairness
- How would you measure the performance of a computer?





Response time vs. throughput



Fairness

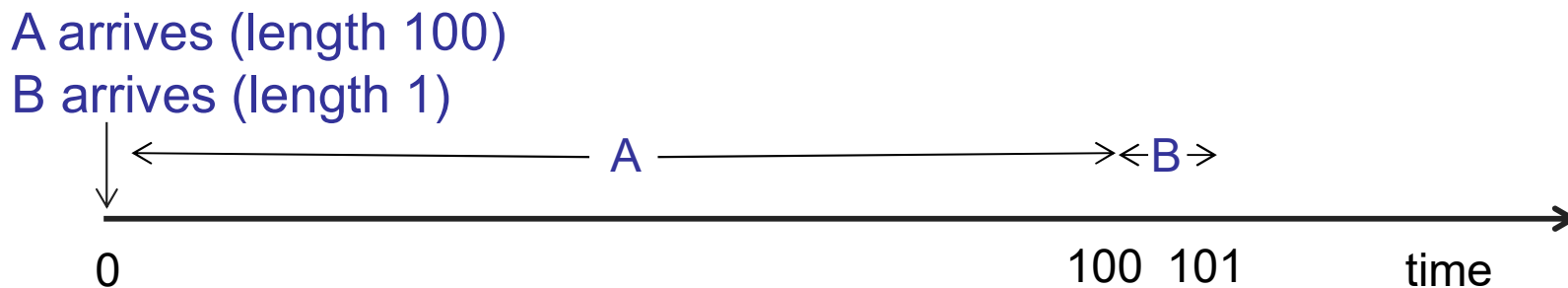
- Second goal of CPU scheduling: share CPU among threads in “fair” manner
- What does “fair” mean?
- Fairness may conflict with performance

Starvation = extremely unfair

- Starvation can be outcome of synchronization
 - Example: Readers can starve writers
- Starvation can also be outcome of scheduling
 - Example: always run highest-priority thread
 - If many high priority threads, low priority starves

First-come, first-served (FCFS)

- FIFO ordering among jobs
- No preemption (no timer interrupts)
 - Thread runs until it calls `yield()` or blocks
- Example



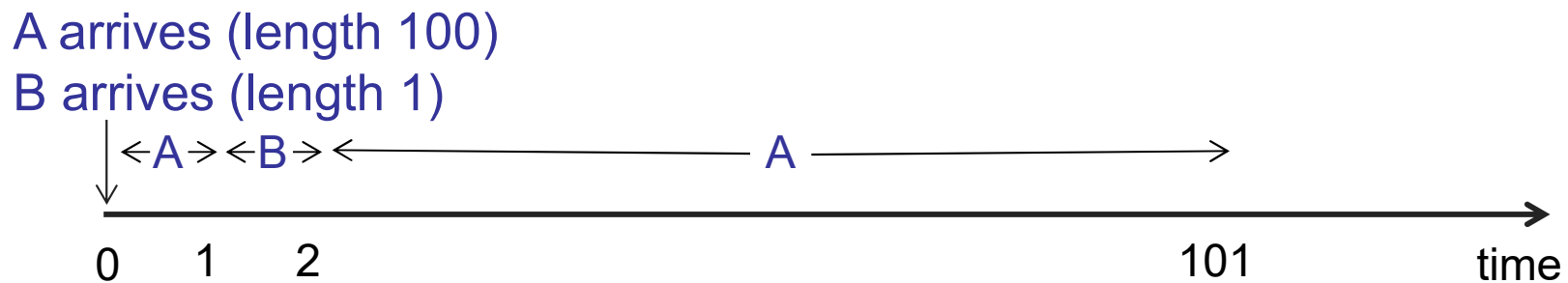
- A: response time = 100
- B: response time = 101
- average response time = 100.5

FCFS summary

- Pros
 - Simple to implement
- Cons
 - Short jobs can be stuck behind long ones
 - Bad for interactive workloads

Round robin

- Goal: Improve average response time for short jobs
- FCFS + periodic preemptions (via timer interrupts)



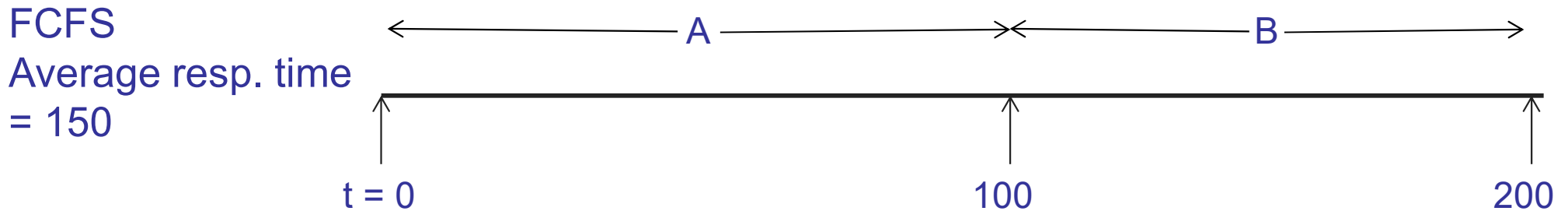
- A: response time = 101
- B: response time = 2
- average response time = 51.5

Round robin

- Round robin achieved lower average response than FCFS in last example
- Does round robin always achieve lower average response time than FCFS?

Round robin vs. FCFS

- Jobs A and B arrive at $t=0$, both take 100 secs



- Which is more “fair”?

Round robin

- Choosing a good time slice
 - What if time slice is too long?
 - What if time slice is too short?

Round robin

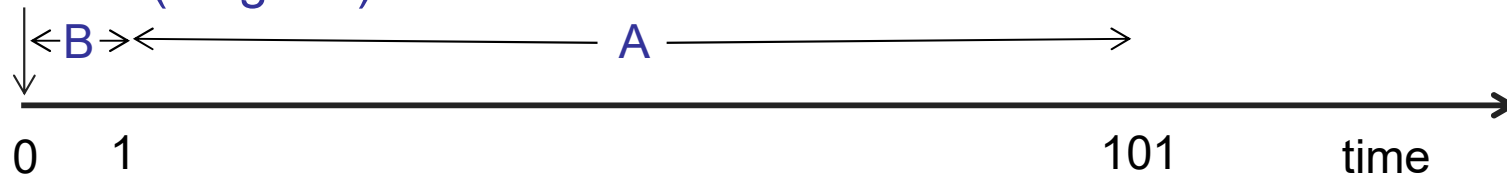
- Pros
 - Faster response time than FCFS for when job lengths vary
 - Often good for interactive computing
- Cons
 - Slower response time than FCFS when job lengths are uniform
 - Additional context switch overhead
- Is there an optimal scheduling policy?

Shortest time to completion first (STCF)

- Run job with least work left to do
 - Improves response time of short jobs (by a lot)
 - Hurts response time of long jobs (by a little)

A arrives (length 100)

B arrives (length 1)



- A: response time = 101
- B: response time = 1
- average response time = 51

STCF is optimal



- If we switch the order of A and B:



- A's response time increases (a little)
 - B's response time decreases (a lot)
 - All other response times remain the same
- ➔ Sum of response times decreases

STCF

- Optimal average response time
- Problems?
- Solutions?

Predicting job run times

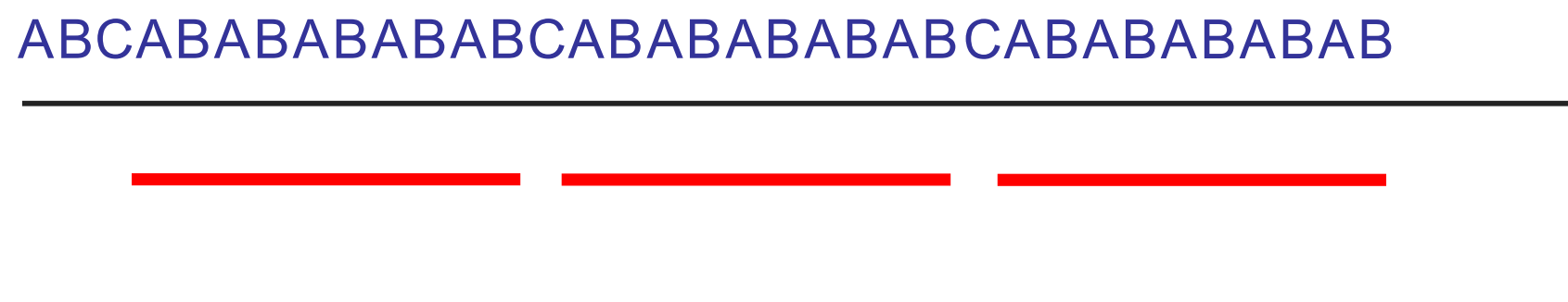
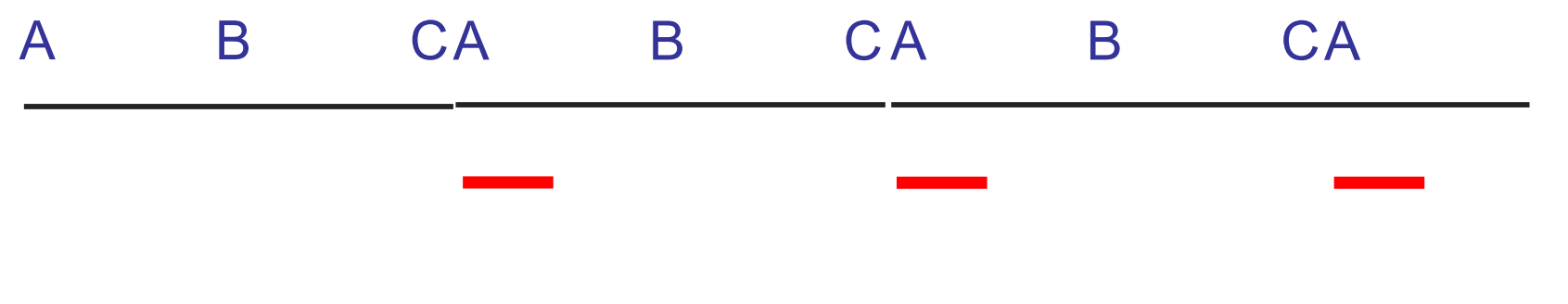
- Ask the user?
 - Incentives?
- Use past to predict future
 - Jobs that have run for a long time will probably continue to run for a long time

Example

- Job A: 100 seconds of CPU
- Job B: 100 seconds of CPU
- Job C:

```
while (1) {  
    1 ms of CPU  
    10 ms of disk  
}
```

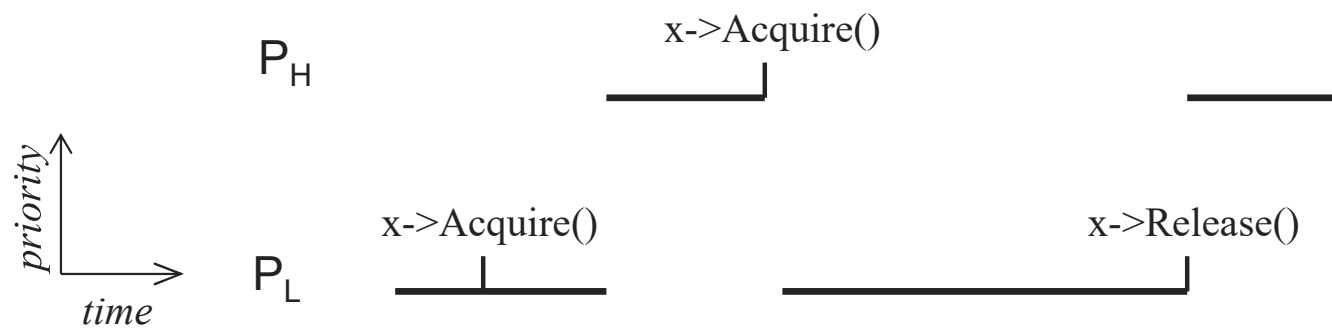
Goal: keep both CPU and disk busy



Priority

- Priority
 - Assign external priority to each job
 - Run high-priority jobs before low-priority ones
 - Use, e.g., round-robin for jobs of equal priority
 - Prone to starvation
- Methods for preventing starvation?
 - If job has not run for time t , boost priority
 - Handle priority inversion (lock held by low-priority)

Priority Inversion



Scheduling in real life

- What scheduling policy do you use for your homework?
- What metric are you optimizing for?

Earliest deadline first (EDF)

- Run the job with the earliest deadline
- Preempt if newly arriving job has earlier deadline
- If schedule is achievable, EDF is optimal!
 - But may be pessimal if schedule is not achievable ☹️

Scheduling: summary

- Many different policies: FCFS, round robin, STCF, EDF, priority, proportional share, etc.
 - OS schedulers use different policies in different settings
- What scheduling strategy is used in grocery stores?
- What scheduling strategy should we use in office hours?

Project 3

- Test cases
 - Enact a path through your state machine. What are the edges?
 - Some edges require multiple processes (e.g., one process evicts another process's page)
 - All groups should test fork of non-empty arena
- Assertions
 - Check that all PTEs are consistent with state of all virtual pages (esp. virtual pages that share a physical resource)
 - Check that swap blocks are accounted for correctly

EECS 482

Introduction to operating systems

File and storage systems

Peter Chen
University of Michigan